SIMATS SCHOOL OF ENGINEERING

Approved by AICTE | IET-UK Accreditation

Course Code: DSA03	**Course Name: Natural Language Processing**
Branch: CSE	**Year: 2026**
Name: Rakshitha V B	**Reg No: 192324004**
Date: 05/11/2026	**Dept: AI&DS**

Assignment 4

1. FOL translations (10M) (CO4) (BLT4)

The following symbols are used in both translations. $\text{Vegetarian}(x)$ says that x is a vegetarian, $\text{Meat}(y)$ says that y is a meat food, $\text{Eats}(x, y)$ says that person x eats food y, and Eggs is a constant for the food item eggs.

a. Vegetarians do not eat meat.

$$\forall x\ \text{Vegetarian}(x) \Rightarrow \forall y\ (\text{Meat}(y) \Rightarrow \neg\text{Eats}(x, y))$$

The sentence is a generalisation over all vegetarians, so the outer quantifier is universal. For a given vegetarian x the claim concerns only those foods y that are meat, which is why Meat(y) is the antecedent of an inner implication and the consequent denies that x eats y. An implication is needed under the universal quantifier because a conjunction would say that every object in the domain is meat.

The same meaning can be written as $\forall x\ \forall y\ (\text{Vegetarian}(x) \wedge \text{Meat}(y)) \Rightarrow \neg\text{Eats}(x, y)$ or as $\neg\exists x\ \exists y\ (\text{Vegetarian}(x) \wedge \text{Meat}(y) \wedge \text{Eats}(x, y))$, which reads as there being no vegetarian who eats any meat. If eating is treated as an event in the Davidsonian style, the sentence becomes:

$$\forall x\ \forall y\ (\text{Vegetarian}(x) \wedge \text{Meat}(y)) \Rightarrow \neg\exists e\ (\text{Eating}(e) \wedge \text{Eater}(e, x) \wedge \text{Eaten}(e, y))$$

b. Not all vegetarians eat eggs.

$$\neg\forall x\ (\text{Vegetarian}(x) \Rightarrow \text{Eats}(x, \text{Eggs}))$$

The sentence denies the universal claim that every vegetarian eats eggs, so the negation stands in front of the quantifier. Pushing it inward changes $\forall$ into $\exists$, and because $\neg(p \Rightarrow q)$ is equivalent to $p \wedge \neg q$, the equivalent form is:

$$\exists x\ (\text{Vegetarian}(x) \wedge \neg\text{Eats}(x, \text{Eggs}))$$

In words, there is at least one vegetarian who does not eat eggs. The sentence does not claim that no vegetarian eats eggs; that would be $\forall x\ \text{Vegetarian}(x) \Rightarrow \neg\text{Eats}(x, \text{Eggs})$, a much stronger statement. Some vegetarians may eat eggs, and the sentence only needs one who does not.

Both translations were checked on three small models. In M1 both sentences are true. In M2 every vegetarian eats eggs, so (b) is false. In M3 one vegetarian eats chicken, so (a) is false.

Output:

```
Vegetarians in every model: Asha, Bala

Model M1 -> Asha eats rice, eggs. Bala eats rice. Chen eats chicken.
  (a) Vegetarians do not eat meat           : True
  (b) Not all vegetarians eat eggs          : True
  (b) Exists vegetarian who skips eggs      : True

Model M2 -> Asha eats rice, eggs. Bala eats rice, eggs. Chen eats mutton.
  (a) Vegetarians do not eat meat           : True
  (b) Not all vegetarians eat eggs          : False
  (b) Exists vegetarian who skips eggs      : False

Model M3 -> Asha eats rice, eggs. Bala eats chicken. Chen eats eggs.
  (a) Vegetarians do not eat meat           : False
  (b) Not all vegetarians eat eggs          : True
  (b) Exists vegetarian who skips eggs      : True
```

2. Facts and inferences (10M) (CO4) (BLT4)

Storing ¬VegetarianRestaurant(McDonalds) or ∃x (Vegetarian(x) ∧ CanEatAt(x, McDonalds)) directly in the knowledge base would prove nothing. Each assertion is therefore derived from general facts about vegetarians, restaurants and food, and the only particular fact about McDonalds is that it is a burger chain. Rules are written R1 to R10, given facts G1 and G2, and derived statements D1 onwards.

a. McDonalds is not a vegetarian restaurant.

A vegetarian restaurant serves only vegetarian food (R5). It is therefore enough to find one item on the McDonalds menu that is not vegetarian. McDonalds is a burger chain, burger chains serve hamburgers, and a hamburger counts as meat, which cannot be vegetarian food.

Label	General knowledge	Meaning
R1	$\forall x\ \text{BurgerChain}(x) \Rightarrow \text{Restaurant}(x)$	A burger chain is a restaurant.
R2	$\forall x\ \text{BurgerChain}(x) \Rightarrow \exists y\ (\text{Hamburger}(y) \wedge \text{Serves}(x, y))$	Every burger chain serves some hamburger.
R3	$\forall y\ \text{Hamburger}(y) \Rightarrow \text{Meat}(y)$	A hamburger is a meat food.
R4	$\forall y\ \text{Meat}(y) \Rightarrow \neg\text{VegetarianFood}(y)$	Meat is not vegetarian food.
R5	$\forall x\ \text{VegetarianRestaurant}(x) \Rightarrow \forall y\ (\text{Serves}(x, y) \Rightarrow \text{VegetarianFood}(y))$	A vegetarian restaurant serves only vegetarian food.
G1	$\text{BurgerChain}(\text{McDonalds})$	McDonalds is a burger chain.

Step	Statement	Justification
D1	Restaurant(McDonalds)	G1, R1, Modus Ponens
D2	Hamburger(h1)	G1, R2, Modus Ponens and existential instantiation with a new constant h1
D3	Serves(McDonalds, h1)	G1, R2, as for D2
D4	Meat(h1)	D2, R3, Modus Ponens
D5	¬VegetarianFood(h1)	D4, R4, Modus Ponens
H1	VegetarianRestaurant(McDonalds)	Assumed, to be refuted
D6	VegetarianFood(h1)	H1, D3, R5 with y = h1, Modus Ponens
D7	⊥	D5 and D6 contradict each other
D8	¬VegetarianRestaurant(McDonalds)	H1 leads to a contradiction, so H1 is rejected

The same result follows from R5 by Modus Tollens: McDonalds serves h1, h1 is not vegetarian food, so the consequent of R5 fails for McDonalds and the antecedent must fail too.

Output:

```
Part (a): McDonalds is not a vegetarian restaurant

  G1  BurgerChain(McDonalds)

  D1   Restaurant(McDonalds)              [R1; from G1]
  D2   Hamburger(h1)                      [R2; from G1]
  D3   Serves(McDonalds, h1)              [R2; from G1]
  D4   Meat(h1)                           [R3; from D2]
  D5   ~VegetarianFood(h1)                [R4; from D4]

  Assume H1: VegetarianRestaurant(McDonalds)
  D6   VegetarianFood(h1)                 [R5; from H1, D3]

  Contradiction: VegetarianFood(h1) and ~VegetarianFood(h1)
  H1 cannot hold, hence ~VegetarianRestaurant(McDonalds)
```

b. Some vegetarians can eat at McDonalds.

Now we need a vegetarian and an item on the McDonalds menu that this vegetarian can eat. McDonalds serves fries, fries are made from potato and are plant based, plant based food is vegetarian food, and a vegetarian can eat any vegetarian food. A person who can eat something that a restaurant serves can eat at that restaurant.

Label	General knowledge	Meaning
R6	$\forall x\ \text{BurgerChain}(x) \Rightarrow \exists y\ (\text{Fries}(y) \wedge \text{Serves}(x, y))$	Every burger chain serves some fries.
R7	$\forall y\ \text{Fries}(y) \Rightarrow \text{PlantBased}(y)$	Fries are plant based.
R8	$\forall y\ \text{PlantBased}(y) \Rightarrow \text{VegetarianFood}(y)$	Plant based food is vegetarian food.
R9	$\forall p\ \forall y\ (\text{Vegetarian}(p) \wedge \text{VegetarianFood}(y)) \Rightarrow \text{CanEat}(p, y)$	A vegetarian can eat vegetarian food.
R10	$\forall p\ \forall y\ \forall r\ (\text{CanEat}(p, y) \wedge \text{Serves}(r, y)) \Rightarrow \text{CanEatAt}(p, r)$	Whoever can eat what a place serves can eat there.
G1	$\text{BurgerChain}(\text{McDonalds})$	McDonalds is a burger chain.
G2	$\text{Vegetarian}(\text{Ravi})$	Ravi is a vegetarian.

Step	Statement	Justification
D1	$\text{Fries}(f1)$	G1, R6, Modus Ponens and existential instantiation with a new constant f1
D2	$\text{Serves}(\text{McDonalds}, f1)$	G1, R6, as for D1
D3	$\text{PlantBased}(f1)$	D1, R7, Modus Ponens
D4	$\text{VegetarianFood}(f1)$	D3, R8, Modus Ponens
D5	$\text{CanEat}(\text{Ravi}, f1)$	G2, D4, R9, Modus Ponens
D6	$\text{CanEatAt}(\text{Ravi}, \text{McDonalds})$	D5, D2, R10, Modus Ponens
D7	$\exists x\ (\text{Vegetarian}(x) \wedge \text{CanEatAt}(x, \text{McDonalds}))$	G2, D6, existential generalisation with x = Ravi

G2 only names one vegetarian, since a sentence about some vegetarians presupposes that at least one exists. Taken together, (a) and (b) fit with each other: McDonalds serves meat and vegetarian dishes, so it is not a vegetarian restaurant, yet vegetarians still have something to eat there.

Output:

```
Part (b): Some vegetarians can eat at McDonalds

  G1  BurgerChain(McDonalds)
  G2  Vegetarian(Ravi)

  D1   Fries(f1)                          [R6; from G1]
  D2   Serves(McDonalds, f1)              [R6; from G1]
  D3   PlantBased(f1)                     [R7; from D1]
  D4   VegetarianFood(f1)                 [R8; from D3]
  D5   CanEat(Ravi, f1)                   [R9; from G2, D4]
  D6   CanEatAt(Ravi, McDonalds)          [R10; from D5, D2]

  Query: exists x. Vegetarian(x) and CanEatAt(x, McDonalds)
  Result: True with x = Ravi
```

3. Temporal relationships (10M) (CO4) (BLT4)

Both sentences describe two events, the departure of Mary's flight and the speaker eating lunch. They differ only in how the two events are placed in time, so each event is represented as a variable with its own time interval, and the relation between intervals is stated separately. The predicates used are Flight(f), Passenger(Mary, f), Departing(e), Departer(e, f), Eating(e), Eater(e, x), Eaten(e, y) and IntervalOf(e, i). Two relations on intervals are needed. Precedes(i, j) holds when i ends before j begins, and Overlaps(i, j) holds when the two intervals share some stretch of time. Now is the time of speaking.

a. When Mary's flight departed, I ate lunch.

$\exists f\, \exists e1\, \exists e2\, \exists i\, \exists j$ (Flight(f) $\wedge$ Passenger(Mary, f) $\wedge$ Departing(e1) $\wedge$ Departer(e1, f) $\wedge$ IntervalOf(e1, i) $\wedge$ Eating(e2) $\wedge$ Eater(e2, Speaker) $\wedge$ Eaten(e2, Lunch) $\wedge$ IntervalOf(e2, j) $\wedge$ Overlaps(i, j) $\wedge$ Precedes(i, Now) $\wedge$ Precedes(j, Now))

The sentence is in the simple past, so both events lie before Now. The word when links the two events: lunch was going on at the moment the flight left. Overlaps(i, j) captures this, because the short departure interval falls inside the longer lunch interval. In Reichenbach's terms the event time and the reference time coincide, and both come before the speech time.

b. When Mary's flight departed, I had eaten lunch.

$\exists f\, \exists e1\, \exists e2\, \exists i\, \exists j$ (Flight(f) $\wedge$ Passenger(Mary, f) $\wedge$ Departing(e1) $\wedge$ Departer(e1, f) $\wedge$ IntervalOf(e1, i) $\wedge$ Eating(e2) $\wedge$ Eater(e2, Speaker) $\wedge$ Eaten(e2, Lunch) $\wedge$ IntervalOf(e2, j) $\wedge$ Precedes(j, i) $\wedge$ Precedes(i, Now))

The past perfect puts lunch earlier than the departure. The departure serves as the reference point, the eating event ended before it, and the reference point itself is before Now, so the order

is lunch, then departure, then speech time. This is why Precedes(j, i) replaces Overlaps(i, j) and the remaining conjuncts stay the same.

Both formulas were evaluated over four small worlds. In W1 lunch is in progress at departure, so only (a) holds. In W2 lunch is over before departure, so only (b) holds. In W3 the flight leaves before lunch and in W4 the flight is not Mary's, so neither sentence holds.

Output:

```
Speech time (Now) = 18:00

W1: Lunch during the departure
    flight owner: Mary  departure 12:30-12:35  lunch 12:00-13:00
    (a) When Mary's flight departed, I ate lunch      : True
    (b) When Mary's flight departed, I had eaten lunch: False

W2: Lunch finished before departure
    flight owner: Mary  departure 14:00-14:05  lunch 12:00-13:00
    (a) When Mary's flight departed, I ate lunch      : False
    (b) When Mary's flight departed, I had eaten lunch: True

W3: Lunch begins after departure
    flight owner: Mary  departure 11:00-11:05  lunch 12:00-13:00
    (a) When Mary's flight departed, I ate lunch      : False
    (b) When Mary's flight departed, I had eaten lunch: False

W4: Lunch during another person's flight
    flight owner: Ravi  departure 12:30-12:35  lunch 12:00-13:00
    (a) When Mary's flight departed, I ate lunch      : False
    (b) When Mary's flight departed, I had eaten lunch: False
```

4. The rule for a vegetarian restaurant (10M) (CO4) (BLT4)

$$\forall x\ \text{VegetarianRestaurant}(x) \Rightarrow \text{Serves}(x, \text{VegetarianFood})$$

The rule is a one-way implication, so it states only a necessary condition: anything that is a vegetarian restaurant serves vegetarian food. As a definition it fails for several reasons.

First, the condition is too weak. A restaurant that sells one salad next to a full menu of meat dishes also serves vegetarian food, so the consequent is true for it and the rule does nothing to stop it from being called a vegetarian restaurant. McDonalds serves fries, so Serves(McDonalds, VegetarianFood) holds and the rule gives no route to the conclusion that McDonalds is not a vegetarian restaurant, which is exactly what question 2(a) needs.

Second, the converse is missing. Nothing in the rule lets us conclude that a restaurant is a vegetarian restaurant, however strictly vegetarian its menu is. For a place like Maharani the truth value of VegetarianRestaurant stays completely open, because the implication is satisfied whether the antecedent is true or false.

Third, the rule does not express what people mean by the phrase. A vegetarian restaurant serves only vegetarian food, not merely some of it, and it must serve something, otherwise a restaurant with an empty menu would qualify vacuously. VegetarianFood is also used as if it were a single individual that can be served, when it is really a category of dishes, so the dishes should be quantified over. Finally, x ranges over everything in the domain and should be limited to restaurants.

A better definition is a biconditional that states both directions and quantifies over the dishes served:

$$\forall x\ \text{VegetarianRestaurant}(x) \Leftrightarrow \text{Restaurant}(x) \wedge \exists y\ \text{Serves}(x, y) \wedge \forall z\ (\text{Serves}(x, z) \Rightarrow \text{VegetarianFood}(z))$$

With this version the left to right direction is the rule R5 used in question 2, and the right to left direction lets us recognise vegetarian restaurants from their menus. The program compares the two versions on five restaurants. Under the old rule a restaurant that serves vegetarian food may be assigned either value, so Maharani, GreenLeaf and McDonalds are all left undecided, while the new definition fixes the value for every restaurant.

Output:

```
Restaurant     Veg dish Non-veg  Old: T ok  Old: F ok  New VR
-----------------------------------------------------------------
Maharani       yes      no       yes        yes        True
GreenLeaf      yes      no       yes        yes        True
McDonalds      yes      yes      yes        yes        False
SteakHouse     no       yes      no         yes        False
ClosedKitchen  no       no       no         yes        False

Old rule leaves VR undecided for: Maharani, GreenLeaf, McDonalds
New definition decides VR for all 5 restaurants
Vegetarian restaurants under new definition: Maharani, GreenLeaf
```